西南交通大學
Southwest Jiaotong University

云计算技术

第10讲 Spark大数据处理

授课教师：戴朋林

课程代码：SCAI004712

2026年5月



00 本讲内容大纲

01 Spark 简介
为什么 Spark 取代大量 MapReduce 工作

02 Spark 架构
Driver / Executor / Cluster Manager / SparkSession

03 RDD
分区、依赖、惰性执行、血缘、缓存

04 DataFrame 与 Spark SQL
结构化处理、Catalyst、Tungsten

05 Spark Streaming
微批、Structured Streaming、Checkpoint

00 本讲内容大纲

01 Spark 简介
为什么 Spark 取代大量 MapReduce 工作

02 Spark 架构
Driver / Executor / Cluster Manager / SparkSession

03 RDD
分区、依赖、惰性执行、血缘、缓存

04 DataFrame 与 Spark SQL
结构化处理、Catalyst、Tungsten

05 Spark Streaming
微批、Structured Streaming、Checkpoint

Spark的设计动机：MapReduce到Spark

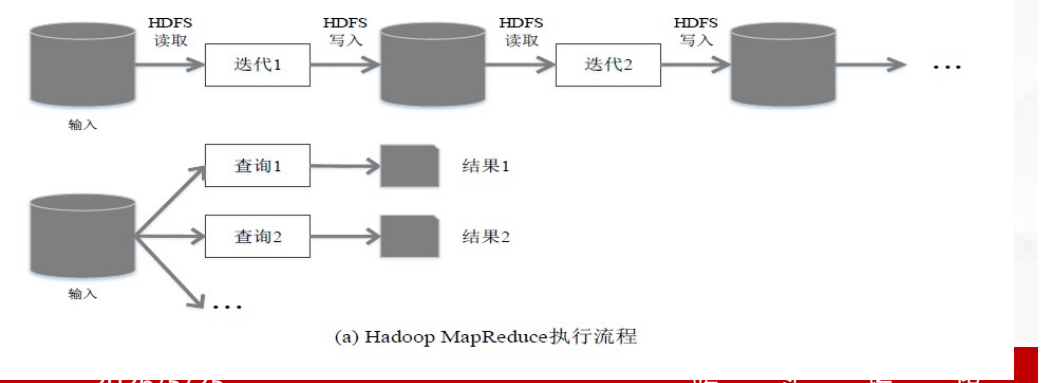
西南交通大学 Southwest Jiaotong University

Hadoop MapReduce 推动了离线大数据处理，但它的执行模型偏“批处理+磁盘中转”

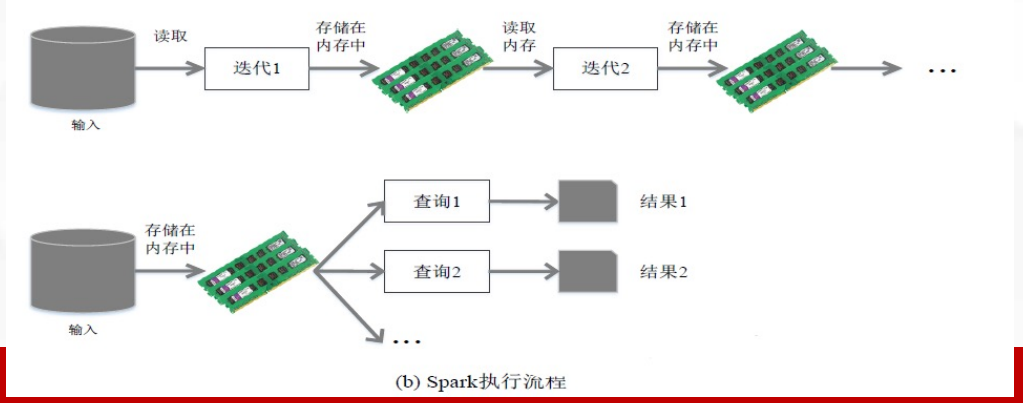
- 对日志清洗这类单次扫描任务，MapReduce 够用
- 对机器学习、交互式查询、图计算等**迭代优化**任务，中间结果反复落盘会非常慢

Spark 核心思想

- 尽量把中间结果保留在内存，减少磁盘 I/O
- DAG执行引擎
- 统一批处理与流处理



(a) Hadoop MapReduce执行流程



(b) Spark执行流程

Spark内存计算 vs MapReduce 磁盘计算

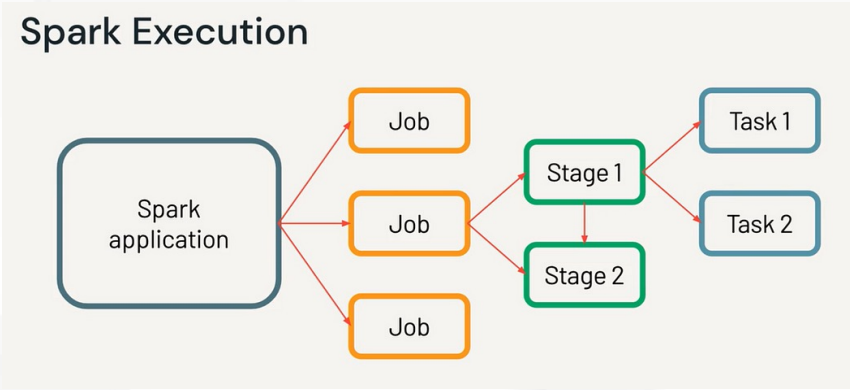
西南交通大学 Southwest Jiaotong University

维度	MapReduce	Spark
中间结果	通常落到本地磁盘/HDFS	尽量保留在内存，必要时再落盘
执行模型	固定 Map→Shuffle→Reduce	DAG + 多算子流水
迭代算法	每轮都重复读写磁盘	可复用缓存数据，迭代效率高
交互式查询	延迟较高	响应更快
典型任务	大规模离线批处理	批处理、SQL、机器学习、流处理

Spark：DAG 执行引擎

西南交通大学 Southwest Jiaotong University

- Spark 把一连串 Transformation 看成一个有向无环图（DAG）
  - 节点是数据集/计算结果
  - 边表示依赖关系
- 执行流程：
  1. 先有逻辑 DAG
  2. 再拆成物理执行 Stage
     - 依据 Shuffle 边界，把 DAG 切成多个 Stage
     - Stage 内部尽量流水执行
- 好处：系统可以观察全局依赖，不只盯着单个 Map 或 Reduce



Spark：DAG 执行引擎

西南交通大学 Southwest Jiaotong University

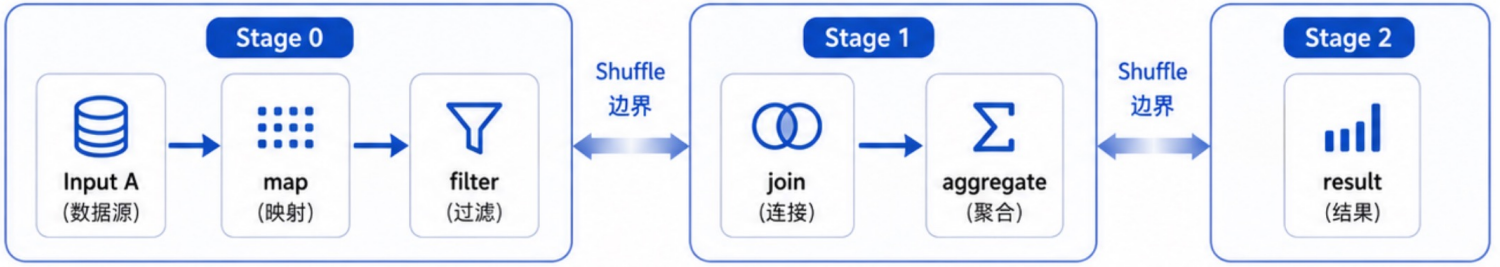
1 构建逻辑DAG

Spark 先根据算子
依赖构建一张逻辑DAG



2 按 Shuffle 边界切分为多个 Stage

遇到 Shuffle（如 join / 聚合）
封装为独立 Stage，
实现更灵活的执行



1. 逻辑DAG

- 基于算子依赖构建完整的逻辑DAG
- 反映作业的全局数据流与依赖关系

2. Stage切分

- 以 Shuffle 为边界切分为多个 Stage
- 每个 Stage 是可独立调度的执行单元

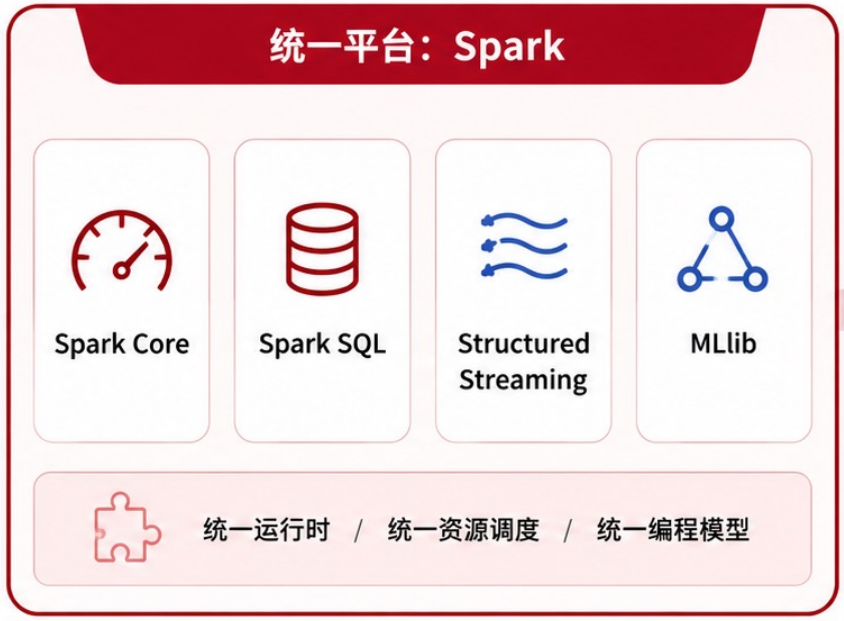
3. 优化价值

- 打破固定两阶段限制，执行更灵活
- 减少不必要的宽依赖，提高并行度
- 更好地支持流水线执行与资源利用

统一批处理与流处理的架构优势

西南交通大学 Southwest Jiaotong University

- **原因：**
  - 离线 ETL、交互式 SQL、机器学习、实时统计通常共享同一批数据
  - 如果每类任务都用不同框架，会增加**学习成本**与**运维成本**
- **优势：**
  - 资源复用
  - 接口统一
  - 降低系统复杂度



Spark 适用场景

西南交通大学 Southwest Jiaotong University

场景	是否适合 Spark	原因
海量离线 ETL	适合	并行处理强，生态成熟
交互式 SQL 分析	适合	DataFrame + Catalyst 性能较好
迭代机器学习	适合	缓存与迭代计算优势明显
毫秒级事务处理	不适合	Spark 不是 OLTP 数据库
超低延迟实时控制	通常不适合	微批模型不一定满足毫秒级要求
超小规模数据脚本	未必需要	本地 Python / Pandas 可能更轻量

00 本讲内容大纲

01 Spark 简介
为什么 Spark 取代大量 MapReduce 工作

02 Spark 架构
Driver / Executor / Cluster Manager / SparkSession

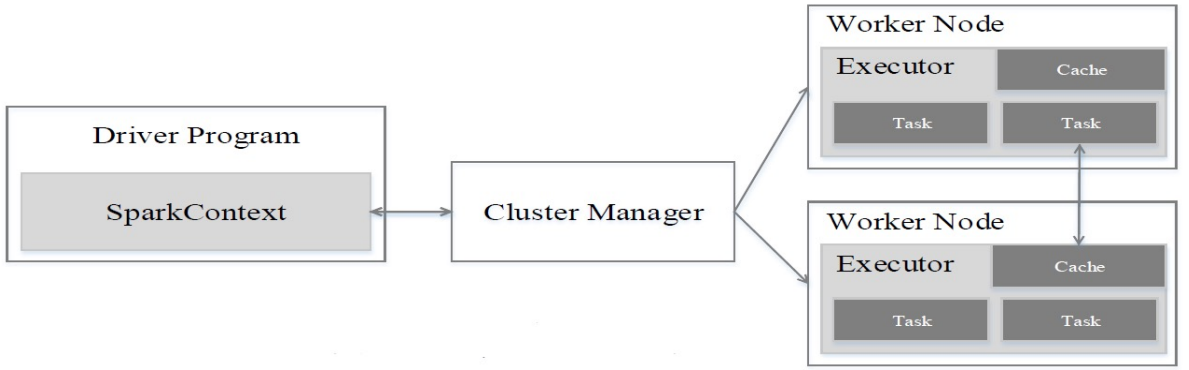
03 RDD
分区、依赖、惰性执行、血缘、缓存

04 DataFrame 与 Spark SQL
结构化处理、Catalyst、Tungsten

06 Spark Streaming
微批、Structured Streaming、Checkpoint

Spark 运行架构

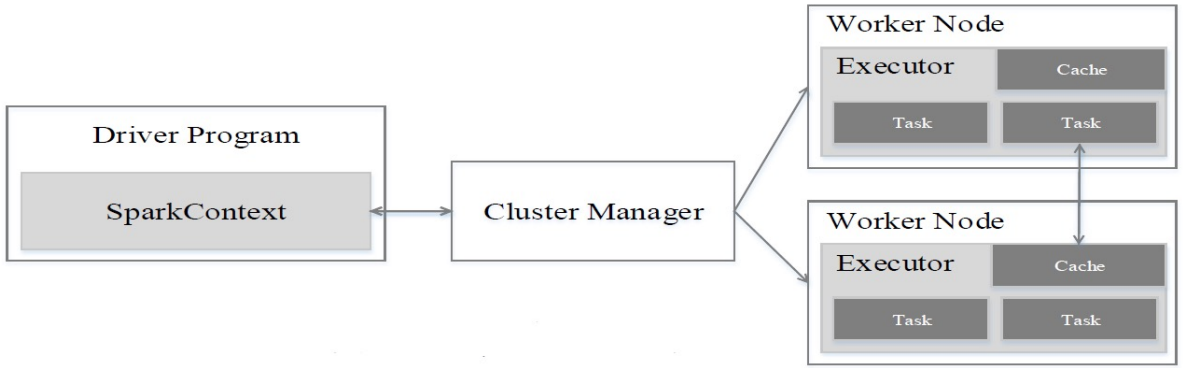
- Spark 运行架构包括：
  - 集群资源管理器 (Cluster Manager)
  - 运行作业任务的工作节点 (Worker Node)
  - 每个应用的任务控制节点 (Driver)
  - 每个工作节点上负责具体任务的执行进程 (Executor)
- 与 Hadoop MapReduce 相比，Spark 所采用的 Executor 有两个优点：
  - 利用多线程来执行具体的任务，减少任务的启动开销。
  - Executor 中有一个 BlockManager 存储模块，会将内存和磁盘共同作为存储设备，有效减少 IO 开销。



Driver Program

西南交通大学 Southwest Jiaotong University

- Driver 是应用的控制中心
  - 运行 main 函数
  - 创建 SparkSession
  - 解析用户写的Transformation / Action
- 当 Action 出现时，Driver 会把逻辑转换成 Job / Stage / Task，并向集群申请资源
- Driver还会维护元信息：例如哪些 RDD 被缓存、哪些 Executor 还活着、任务失败如何重试



Executor的工作模式：长期驻留

西南交通大学 Southwest Jiaotong University

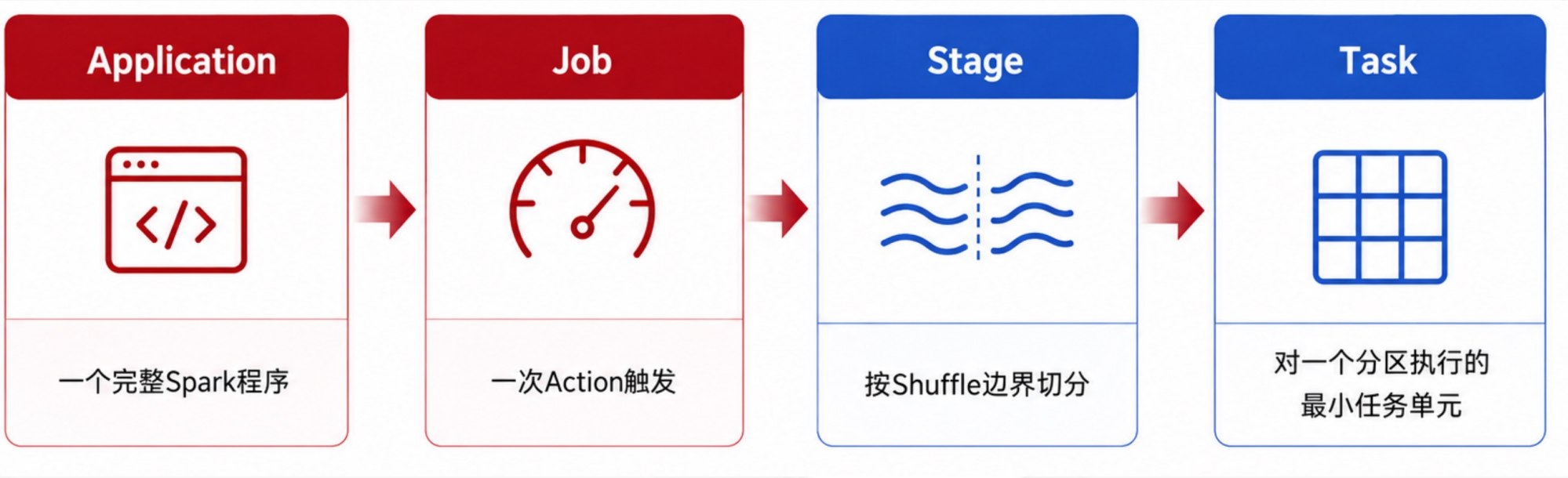
- Executor 是运行在工作节点上的进程
  - 负责接收 Driver 下发的 Task 并实际执行
  - 管理本地 Cache 与 Shuffle 数据。

- 长期驻留的优势：
  - 如果每个小任务都创建/销毁一次进程，分布式调度开销会很大
  - 长期驻留 Executor，而不是每个任务临时起进程，可以减少频繁启动进程的开销。

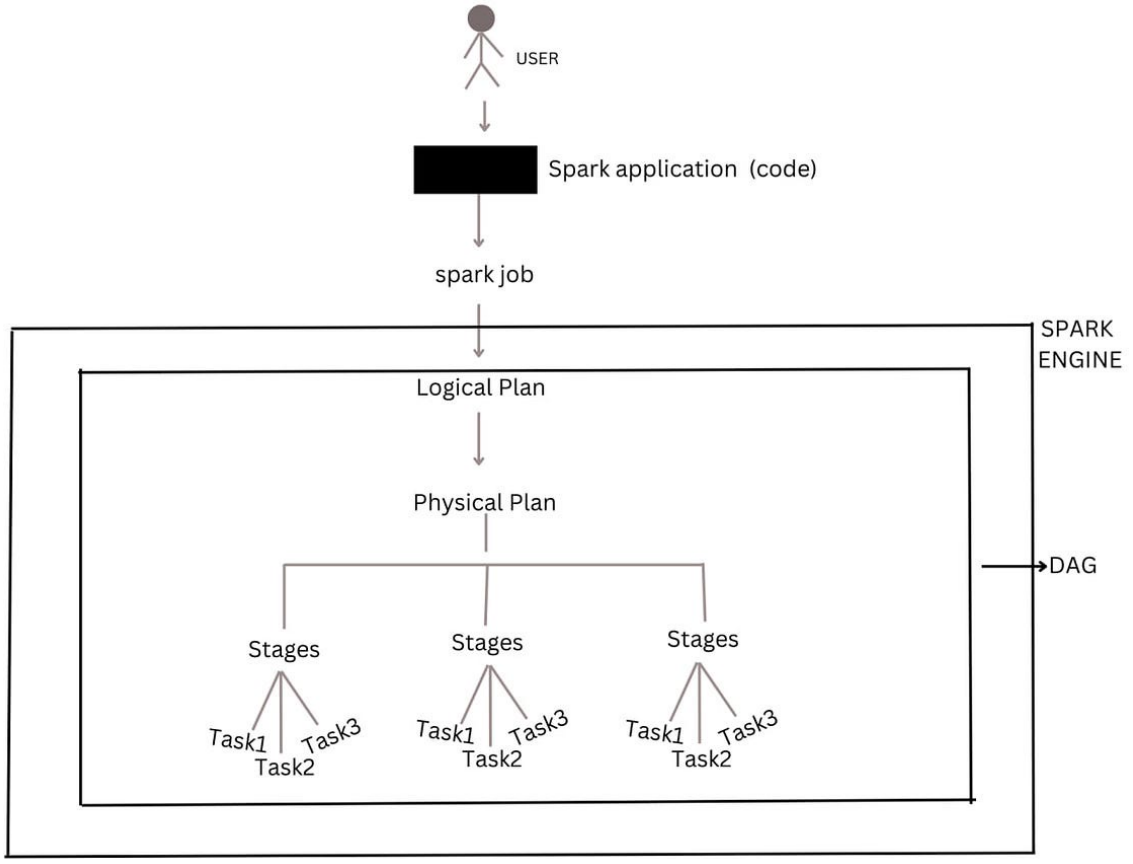
Executor的工作模式：作业执行层级

西南交通大学 Southwest Jiaotong University

分层抽象能把用户程序一步步映射到机器执行单元

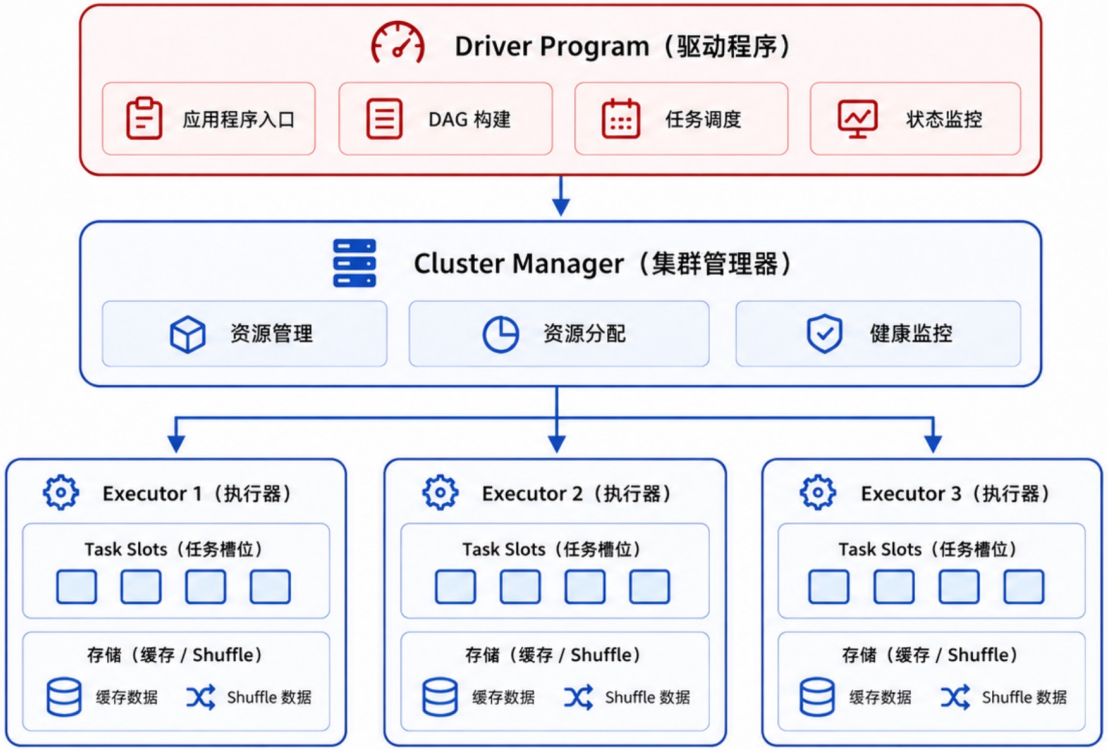


Executor的工作模式：作业执行层级



Cluster Manager

- 为什么需要Cluster manager：
  - Spark 本身专注于计算逻辑
  - 集群资源管理与节点生命周期
- 好处：
  - 可以运行在多种环境：Standalone、Hadoop YARN、Mesos、Kubernetes
  - 资源管理层与计算引擎解耦后，Spark的部署灵活性更强



Cluster Manager类型

西南交通大学 Southwest Jiaotong University

- 同样的 Spark 程序，可能跑在学校实验集群、企业 Hadoop 集群或云上 K8s
- 抽象多种资源后端是为了适配不同基础设施

类型	特点	适用场景
Standalone	Spark 自带，部署简单	教学、小型集群、快速实验
YARN	与 Hadoop 生态集成紧密	已有 HDFS/Hadoop 基础设施的企业
Mesos	通用资源调度平台	多框架共享集群
Kubernetes	容器化、弹性伸缩、云原生	现代云环境与容器平台

SparkSession作为统一入口：API统一化

西南交通大学 Southwest Jiaotong University

- 问题：
  - 早期 Spark 有 SparkContext、SQLContext、HiveContext 等多个入口
  - 当功能模块越来越多时，如果入口过于分裂，使用门槛会明显上升
- 解决方法：
  - SparkSession 统一了 RDD、DataFrame、SQL 等能力。
  - 通过 SparkSession.builder 配置 appName、master 与运行参数
  - 统一入口让 API 更一致，也程序结构更清晰

00 本讲内容大纲

01 Spark 简介
为什么 Spark 取代大量 MapReduce 工作

02 Spark 架构
Driver / Executor / Cluster Manager / SparkSession

03 RDD
分区、依赖、惰性执行、血缘、缓存

04 DataFrame 与 Spark SQL
结构化处理、Catalyst、Tungsten

05 Spark Streaming
微批、Structured Streaming、Checkpoint

RDD（Resilient Distributed Dataset）

西南交通大学 Southwest Jiaotong University

- RDD 是 Spark 最基础的数据抽象
- 用户不必担心底层数据的分布式特性，只需将应用逻辑表达为一系列转换处理
- 不同 RDD 之间的转换操作形成依赖关系，可以实现管道化，避免中间数据存储
- RDD的五大特性：分区、依赖、计算函数、分区器、首选位置

特性	含义	作用
Partitions	分区集合	决定并行度与任务划分
Dependencies	与父 RDD 的依赖	决定 Stage 与容错恢复
Compute Function	每个分区如何计算	定义转换逻辑
Partitioner	键值数据如何分区	优化聚合与连接
Preferred Locations	首选位置	提高数据本地性

Partition

西南交通大学 Southwest Jiaotong University

Partition：

- RDD 会被切成多个分区，每个分区通常对应一个 Task
- 分区数越少，并行度可能不足；分区数过多，调度开销又会上升
- 合理分区有助于均衡负载、提高资源利用率

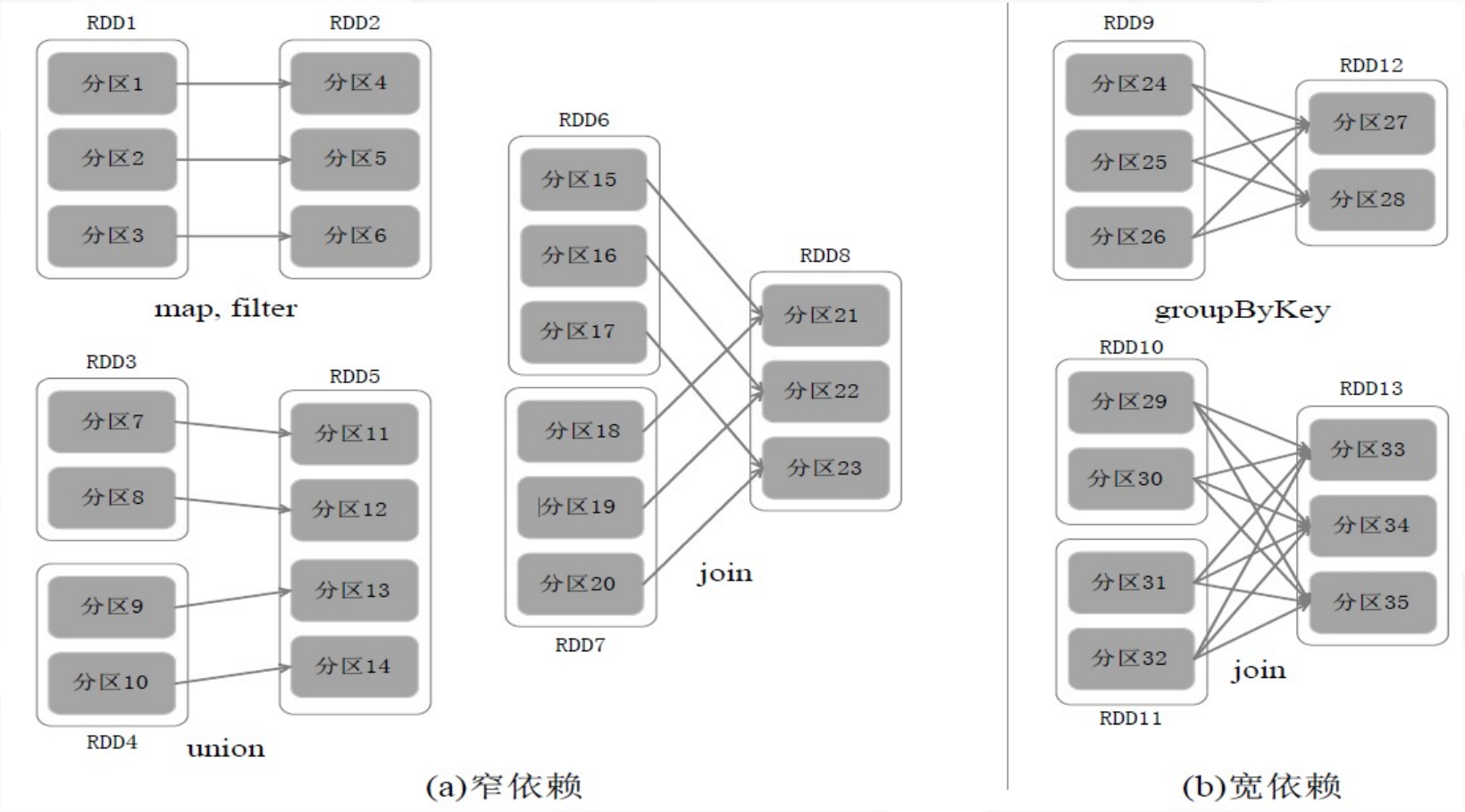
Dependency

西南交通大学 Southwest Jiaotong University

Dependency：

- RDD 之间通过 **Dependency** 连接
- 两种依赖：
  - **窄依赖**： 子分区只依赖少量父分区，适合流水执行，如 map、filter
  - **宽依赖**：父 RDD 的一个分区对应一个子 RDD 的多个分区。可能产生 Shuffle，如 reduceByKey、join、groupByKey
- Spark 根据依赖关系切分 Stage 并进行容错恢复

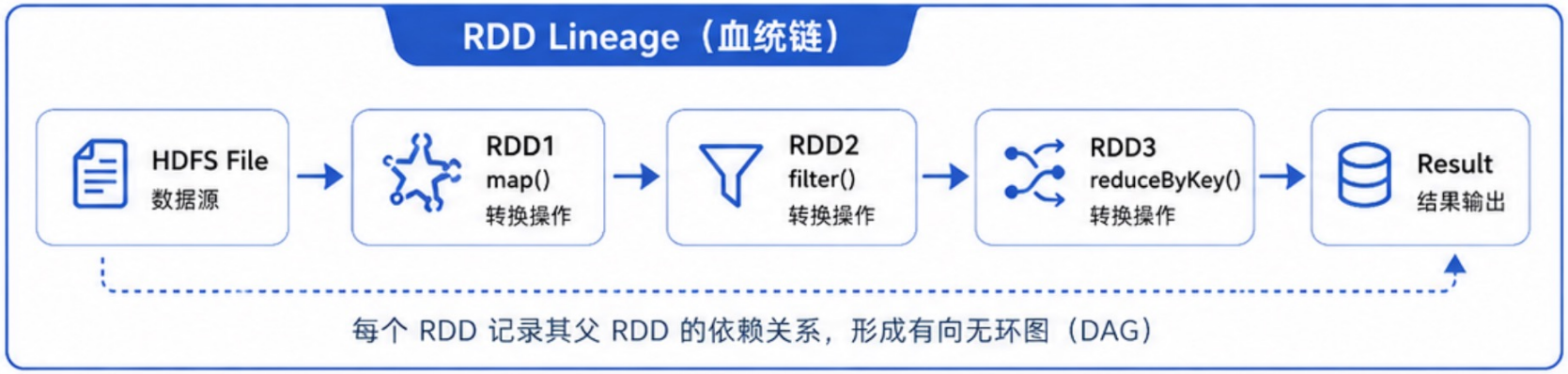
Dependency


RDD1
RDD2
分区1
分区2
分区3
分区4
分区5
分区6
map, filter
RDD3
RDD5
分区7
分区8
分区9
分区10
分区11
分区12
分区13
分区14
RDD4
union
RDD6
分区15
分区16
分区17
分区18
分区19
分区20
RDD7
RDD8
分区21
分区22
分区23
join
(a)窄依赖
RDD9
分区24
分区25
分区26
RDD12
分区27
分区28
groupByKey
RDD10
分区29
分区30
分区31
分区32
RDD11
RDD13
分区33
分区34
分区35
join
(b)宽依赖

西南交通大学 Southwest Jiaotong University

Compute Function

- 每个RDD分区如何计算
  - 每个 RDD 记录如何从父 RDD 的分区计算得到当前分区
  - **RDD 不可变**：每次 Transformation 都返回新 RDD，让血缘链更清晰，降低修改复杂性



Partitioner

西南交通大学 Southwest Jiaotong University

键值数据怎么分配?

- Partitioner 决定某个 Key 落到哪个分区
- Pair RDD 可以使用 HashPartitioner 或 RangePartitioner
- 相同分区器可以减少某些 join / aggregate 的 Shuffle
- 本质上是针对键值聚合与连接的性能优化

Preferred Locations

西南交通大学 Southwest Jiaotong University

为什么靠近数据?

- 如果分区原本在某个节点，Task 最好调度到附近执行。
- **数据本地性**可以减少跨网络传输
- Spark 会在本地性和资源等待之间做平衡
- 大数据环境中，搬计算通常比搬数据更便宜

Transformation 与 Action

西南交通大学 Southwest Jiaotong University

计算功能和触发功能分离

类别	代表操作	特点
Transformation	map、filter、flatMap、reduceByKey	返回新 RDD，通常惰性执行
Action	collect、count、take、saveAsTextFile	触发真正计算

常见 Transformation

西南交通大学 Southwest Jiaotong University

操作	作用	课堂说明
map	一对一映射	元素级转换
filter	条件过滤	保留满足条件元素
flatMap	一对多展开	常用于分词
distinct	去重	通常涉及 Shuffle
groupByKey	按 Key 分组	可能代价较高
reduceByKey	按 Key 聚合	先局部聚合再全局聚合

常见 Action

西南交通大学 Southwest Jiaotong University

操作	作用	课堂说明
collect	拉回全部结果到 Driver	小数据可用，大数据慎用
count	统计记录数	返回值小，常用于验证规模
take(n)	取前 n 条	调试常用
first	取第一条	快速检查数据
saveAsTextFile	输出到文件系统	离线结果落盘

PySpark RDD 示例：WordCount

西南交通大学 Southwest Jiaotong University

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("RDDWordCount") \
    .master("local[*]") \
    .getOrCreate()

sc = spark.sparkContext
lines = sc.textFile("data/words.txt")
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda w: (w.lower(), 1))
counts = pairs.reduceByKey(lambda a, b: a + b)

print(counts.collect())
spark.stop()
```

- **textFile** 读取文本形成 RDD。
- **flatMap** 把一行拆成多个单词。
- **map** 构造 (word, 1) 键值对。
- **reduceByKey** 完成分布式聚合。

PySpark RDD 示例：值对统计

西南交通大学 Southwest Jiaotong University

```python
data = [("A", 10), ("A", 20), ("B", 5), ("C", 8), ("B", 3)]
rdd = sc.parallelize(data, 2)

sum_rdd = rdd.reduceByKey(lambda x, y: x + y)
filtered = rdd.filter(lambda kv: kv[1] >= 8)

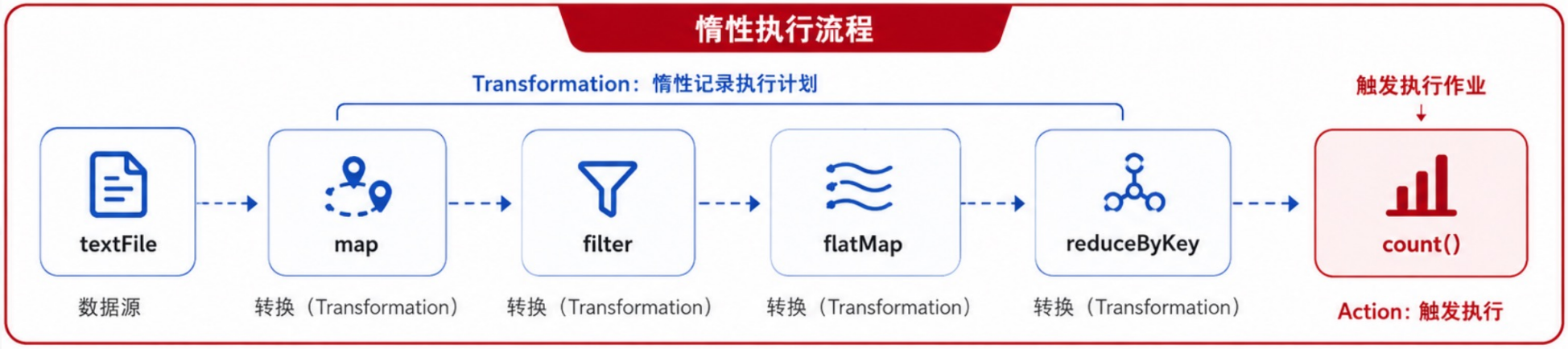
print("sum =", sum_rdd.collect())
print("filtered =", filtered.collect())
```

- **parallelize** 把本地集合并行化为 RDD。
- **reduceByKey** 对相同 Key 的值做聚合。
- **filter** 用条件筛选记录。
- Pair RDD 是实际业务中很常见的数据形式。

惰性执行（Lazy Evaluation）

西南交通大学 Southwest Jiaotong University

- 惰性执行：Transformation 默认不会立即运行，它们只会构建一条逻辑计算链；Spark 等 Action 出现后统一执行
- 带来的好处：
  - 减少无意义中间结果
  - 给优化器留出全局优化空间
  - Spark 能做 DAG 优化与 Stage 切分的前提



缓存策略：Cache和Persist

西南交通大学 Southwest Jiaotong University

- Cache() ≈ Persist(MEMORY_ONLY)
- Persist()： 支持更多存储级别
  - MEMORY_ONLY
  - MEMORY_AND_DISK
  - DISK_ONLY
- 数据复用越多，缓存收益越明显



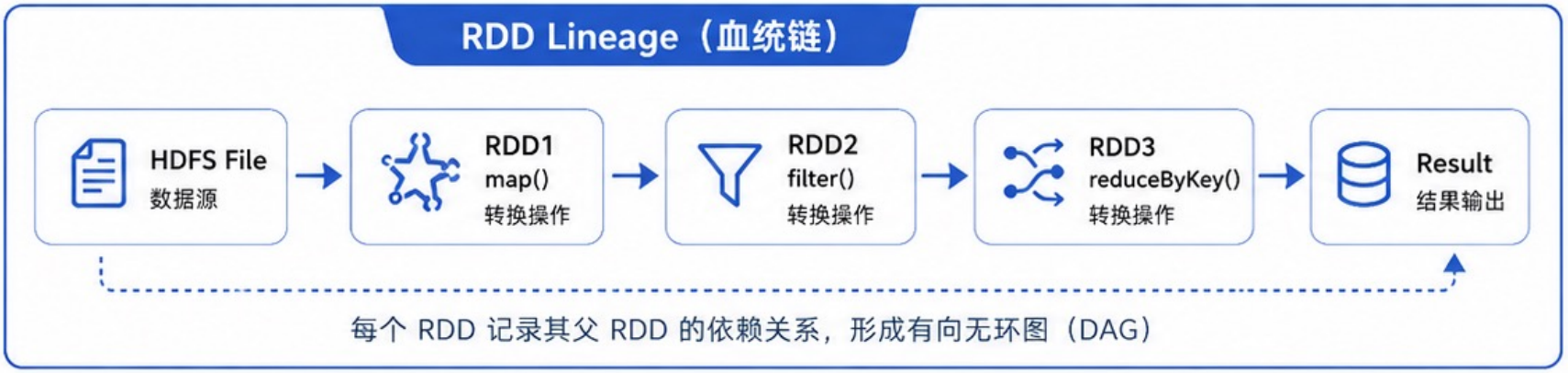




血缘追踪（Lineage）与容错

西南交通大学 Southwest Jiaotong University

- Spark 为 RDD 记录 **Lineage（血缘）**，而不是对每个中间结果都进行复制
- 一旦某个分区丢失，就沿着血缘链重新计算该分区
- **全量复制→按需重算**
- 如果某些数据代价太高，通过 Checkpoint 持久化提升恢复稳定性。



00 本讲内容大纲

01 Spark 简介
为什么 Spark 取代大量 MapReduce 工作

02 Spark 架构
Driver / Executor / Cluster Manager / SparkSession

03 RDD
分区、依赖、惰性执行、血缘、缓存

04 DataFrame 与 Spark SQL
结构化处理、Catalyst、Tungsten

05 Spark Streaming
微批、Structured Streaming、Checkpoint

结构化数据处理的优势

西南交通大学 Southwest Jiaotong University

为什么需要 DataFrame 与 Spark SQL?

- RDD 很灵活，但系统不知道字段名、类型和列结构，难以做高级自动优化
- **DataFrame** = 带 Schema 的分布式表
- **Spark SQL** 让用户用 SQL 描述结构化查询

name	age	city
Alice	23	Beijing
Bob	30	Shanghai
Cathy	28	Shenzhen
…	…	…



Schema（结构）

```
name: string
age: int
city: string
```

DataFrame = 数据 + 结构信息

DataFrame常用API

西南交通大学 Southwest Jiaotong University

操作	作用	示例
select	列选择/表达式	df.select("name", "age")
filter/where	行过滤	df.filter(df.age > 20)
groupBy	分组聚合	df.groupBy("city").count()
orderBy	排序	df.orderBy(df.age.desc())
join	表连接	df1.join(df2, "id")

Spark SQL 查询示例

西南交通大学 Southwest Jiaotong University

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("SQLDemo").getOrCreate()

data = [("Alice", 21, "CS"), ("Bob", 22, "SE"), ("Cici", 21, "CS")]
df = spark.createDataFrame(data, ["name", "age", "major"])

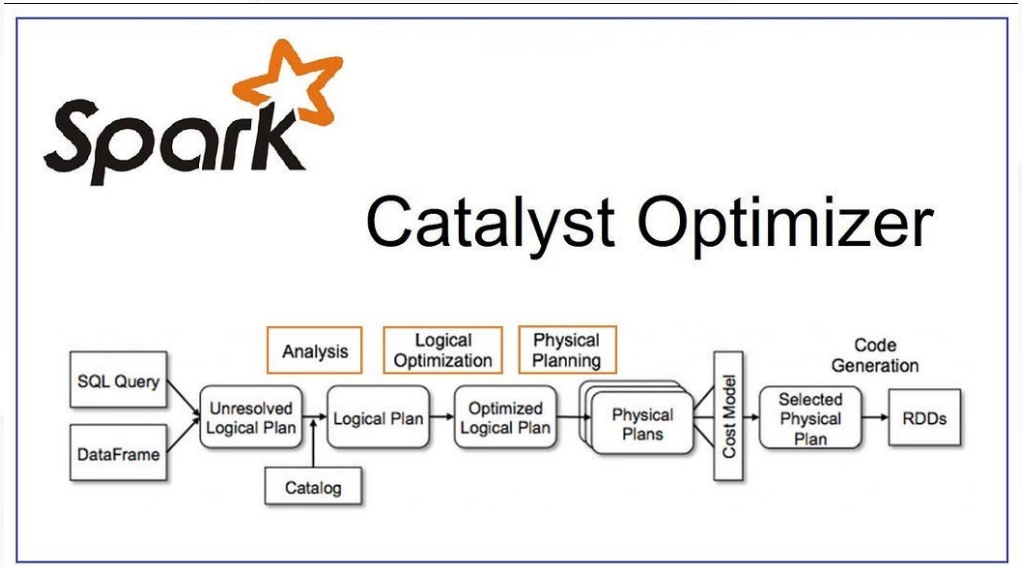
df.createOrReplaceTempView("students")
result = spark.sql("""
    SELECT major, COUNT(*) AS cnt, AVG(age) AS avg_age
    FROM students
    GROUP BY major
    ORDER BY cnt DESC
""")
result.show()
```

- **createOrReplaceTempView** 注册临时视图，把 DataFrame 注册为临时视图 students。
- 随后可以使用 标准SQL语法查询。
- SQL 对用Spark进行分析的人员门槛降低

查询优化器Catalyst

西南交通大学 Southwest Jiaotong University

- **Catalyst** 是 Spark SQL 的查询优化器
- Catalyst 的基本工作流程
  1. Logical Plan Creation
  2. Logical Plan Optimization
  3. Physical Plan Generation
  4. ~~Code Generation~~



查询优化器Catalyst

西南交通大学 Southwest Jiaotong University

- Step 2: Logical Plan Optimization
- 常见操作：
  - Push down filters
  - Combine operations
- 目的：提升优化计算效率

```
# Show the optimized logical plan
filtered_df.explain(True)
```

查询优化器Catalyst

西南交通大学 Southwest Jiaotong University

- Step 3: Physical Plan Generation
- 将逻辑计划转换为机器执行计划
- 根据数据、资源等状态，优化执行效率

```
# Physical plan explanation
filtered_df.explain()
```

```
summary.explain(mode="formatted")
```

```
# == Physical Plan ==
# AdaptiveSparkPlan
# +- Sort
#    +- HashAggregate
#       +- Exchange
#          +- HashAggregate
#             +- Filter
#                +- FileScan csv ...
```

- FileScan：读取数据。
- Filter：过滤操作。
- Exchange：发生 Shuffle。
- HashAggregate：聚合物理算子。

Tungsten内存管理与代码生成

西南交通大学 Southwest Jiaotong University

- Step 4： Code Generation

- 好计划还不够，执行层也要足够高效
- **Tungsten**的职责
  - Spark内存管理
  - 堆外内存（off-heap memory） 管理
  - 代码生成

00 本讲内容大纲

01 Spark 简介
为什么 Spark 取代大量 MapReduce 工作

02 Spark 架构
Driver / Executor / Cluster Manager / SparkSession

03 RDD
分区、依赖、惰性执行、血缘、缓存

04 DataFrame 与 Spark SQL
结构化处理、Catalyst、Tungsten

05 Spark Streaming
微批、Structured Streaming、Checkpoint

为什么需要Spark Streaming?

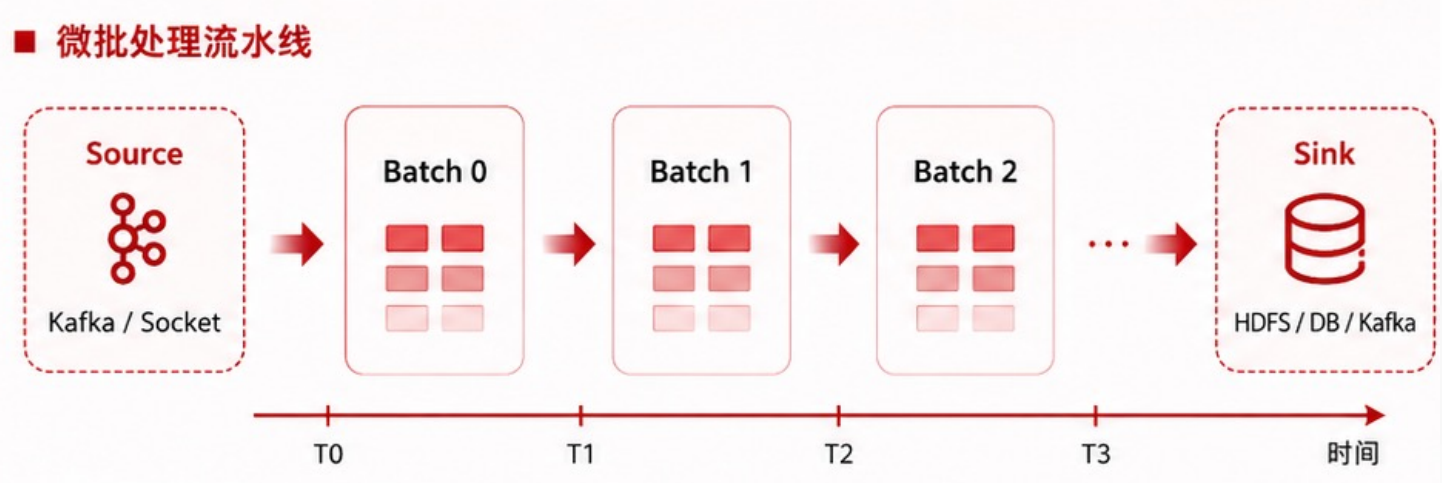
西南交通大学 Southwest Jiaotong University

- 日志、点击流、传感器数据通常持续不断地产生
- 如果只做离线批处理，只能看到历史快照
- 业务希望边产生数据边分析，流处理Streaming就成为必要

微批处理（micro-batch）模型

西南交通大学 Southwest Jiaotong University

- Micro-batch：把连续的数据流按时间片切分为一批批的小任务
- 每个批次的小任务≈RDD
- 可以复用Spark批处理引擎



窗口操作

西南交通大学 Southwest Jiaotong University

- 窗口操作：对一个时间窗口内的micro-batches做聚合
- 为什么要使用窗口：关注一段时间的性能/数据
- 常见参数：窗口大小、滑动间隔
- 例：每 10 秒统计最近 1 分钟错误日志数
  - 窗口大小：1分钟
  - 滑动间隔：10秒



Structured Streaming 代码示例：实时词频统计

西南交通大学 Southwest Jiaotong University

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode, split

spark = SparkSession.builder.appName("StructuredStreamingWC").getOrCreate()
lines = (spark.readStream
    .format("socket")
    .option("host", "localhost")
    .option("port", 9999)
    .load())

words = lines.select(explode(split(lines.value, " ")).alias("word"))
counts = words.groupBy("word").count()

query = (counts.writeStream
    .outputMode("complete")
    .format("console")
    .option("checkpointLocation", "chk/wc")
    .start())

query.awaitTermination()
```

- readStream 建立流式数据源。
- select / groupBy / count 与批处理写法相似
- writeStream 指定输出模式和 Sink。
- checkpointLocation 用于故障恢复。

Structured Streaming 的优势是用类似批处理的 API 处理无界数据流。

Checkpoint：流任务如何容错?

西南交通大学 Southwest Jiaotong University

- 流式任务通常长时间运行，故障后不能简单从头再来
- **Checkpoint**：状态和元数据持久化到可靠存储
- 用途：
  - 任务重启恢复状态
  - 故障恢复
  - 窗口统计
  - 有状态计算



课堂思考题

西南交通大学 Southwest Jiaotong University

1. 为什么 Spark 在迭代机器学习任务上通常比 MapReduce 更有优势?
2. 为什么 reduceByKey 往往比 groupByKey 更合适大规模聚合?
3. 如果一个 DataFrame 查询很慢，你会从 Spark UI 与 explain() 中先看什么?
4. 什么场景下应选择 persist(MEMORY_AND_DISK)，而不是 cache()?
5. 为什么 Structured Streaming 更符合 Spark 的长期演进方向?